

Во рамки на оваа домашна задача, апликацијата беше рефакторирана со цел да се подобри нејзината јасност, одржливост, повторна употребливост и проширливост, преку примена на софтверски шаблони (Design Patterns) дискутирани на предавањата.

При рефакторирањето беа имплементирани три Gang of Four (GoF) софтверски шаблони:

Observer Pattern, Factory Pattern (Simple Factory) и Strategy Pattern.

Целта на ова рефакторирање беше да се отстрани дупликатен код, да се подобри поделбата на одговорности и да се овозможи лесно додавање на нови функционалности без модификација на постоечкиот код.

1. Observer Pattern

Observer Pattern е имплементиран во делот за обработка на податоци (data pipeline), каде што е потребно повеќе компоненти да бидат известени кога ќе се ажурираат податоците за цените на криптовалути.

Кога нови ценовни податоци се внесуваат во базата преку `filter3_update.py`, објектот `CryptoDataSubject` автоматски ги известува сите регистрирани набљудувачи (observers), како што се:

- поништување на застарени кеширани податоци,
- логирање на аналитички информации,
- проверка за значајни промени во цените.

Со ова:

- логиката за ажурирање на податоци е одвоена од логиката која реагира на тие промени,
- нови функционалности можат да се додаваат како нови observers без промена на основниот код,
- се почитува Open/Closed Principle.

Овој пристап го прави кодот пофлексибилен, полесен за одржување и повторно употреблив.

2. Factory Pattern (Simple Factory)

Factory Pattern е применет во `analysis-service` делот на апликацијата, каде што се креираат различни технички индикатори.

Наместо индикаторите да се креираат со повеќе повторувачки `try-except` блокови низ кодот, е имплементирана `IndicatorFactory` класа која:

- централизирано ја содржи целата логика за креирање индикатори,
- обезбедува конзистентно ракување со грешки,
- значително го намалува дупликатниот код.

Со ова рефакторирање:

- кодот е јасен и доследен,
- се применува DRY (Don't Repeat Yourself) принципот,
- додавање нов индикатор бара минимална промена (само проширување на factory методот),
- се подобрува читливоста и одржливоста на апликацијата.

3. Strategy Pattern

Strategy Pattern е користен за имплементација на различни алгоритми за пресметка на трговски сигнали, како што се:

- RSI,
- MACD,
- Stochastic,
- ADX,
- CCI,
- Moving Averages,
- Bollinger Bands,
- Volume-based сигнали.

Секој алгоритам е имплементиран како посебна стратегија која го имплементира заедничкиот интерфејс SignalStrategy, со метод calculate_signal() што враќа "BUY", "SELL" или "HOLD".

Ова овозможува:

- лесна замена или додавање на нови стратегии без промена на постоечкиот код,
- елиминација на повторен код (проверки за NaN вредности, грешки, итн.),
- подобра организација и повторна употребливост на кодот.

Strategy Pattern овозможува логиката за пресметка на сигнали да варира независно од делот од апликацијата кој ја користи.